

HiGHS 双対単体法 (HEkkDual) の実装まとめ

HiGHS ERGO-Code/HiGHS `highs/simplex/HEkkDual*.{h,cpp}`

2026 年 9 月 12 日

概要

本稿は、オープンソース線形計画ソルバ HiGHS (<https://github.com/ERGO-Code/HiGHS>) の双対単体法実装 (クラス `HEkkDual` / `HEkkDualRow` / `HEkkDualRHS`, データ本体 `HEkk`) について、実際のソースコード (`highs/simplex/` 以下, 2026 年時点の `master` ブランチ) を読みながら、用いられているデータ構造と各手続きの数式的な意味を整理する。ENOMOTO-Solver 独自の有界変数専用双対単体法 (別稿参照) と異なり、HiGHS は自由変数・両側範囲制約行 (`ranged row`)・退化に対する複数の対策 (コスト摂動, 双対フェーズ 1 の人工上下限, Devex/DSE 切り替え, 特異基底からのバックトラック) を備えた汎用実装であり、随所でその違いにも触れる。

目次

1	全体アーキテクチャ	1
2	標準形とデータ構造	2
2.1	LP の内部表現: 範囲制約行と論理変数	2
2.2	基底: 3 本の配列 <code>SimplexBasis</code>	2
2.3	作業配列 <code>HighsSimplexInfo</code> (<code>SimplexStruct.h</code>)	3
2.4	DSE 重みと <code>HEkkDualRow</code> / <code>HEkkDualRHS</code> の作業領域	3
3	フェーズ 1 / フェーズ 2	4
3.1	開始前の判定: <code>HEkkDual::solve</code>	4
3.2	フェーズ 1: 人工上下限による「双対不能性の一次不能性への変換」	4
3.3	コスト摂動: <code>HEkk::initialiseCost</code>	4
4	メインループ: <code>HEkkDual::iterate</code>	5
4.1	<code>CHUZR::chooseRow</code> と <code>HEkkDualRHS::chooseNormal</code>	5
4.2	<code>PRICE</code> と <code>CHUZR::chooseColumn</code> と <code>HEkkDualRow</code>	6
4.3	<code>FTRAN</code> 群: <code>updateFtranBFRT</code> / <code>updateFtran</code> / <code>updateFtranDSE</code>	7
4.4	数値検証: <code>updateVerify</code>	7
4.5	双対値の更新: <code>updateDual</code>	7
4.6	主変数と DSE/Devex 重みの更新: <code>updatePrimal</code>	8
4.7	基底交換: <code>updatePivots</code>	8
5	Devex と DSE の切り替え, および安定化機構のまとめ	8

1 全体アーキテクチャ

HiGHS の単体法は 1 つの巨大関数ではなく、役割ごとに分割された複数のクラスが HEkk (LP・基底・作業配列を保持する「データ本体」) への参照を共有する形で構成されている。

- HEkk (HEkk.h/.cpp): LP 本体 (lp_), 基底 (basis_), 作業配列一式 (info_, 型 HighsSimplexInfo), DSE 重み (dual_edge_weight_), 基底の invertible 表現 (simplex_nla_, LU 分解+更新) を保持する。主/双対単体法の両方から共有される。
- HEkkDual (HEkkDual.h/.cpp, 本稿の主対象): 双対単体法のアルゴリズム本体。solve/solvePhase1/solvePhase2/iterate など。
- HEkkDualRow (HEkkDualRow.h/.cpp): ピボット行の格納, CHUZC (進入列選択) と bound-flipping ratio test (BFRT)。
- HEkkDualRHS (HEkkDualRHS.h/.cpp): CHUZR (退出行選択) と一次実行不能性リストの保守。

ENOMOTO-Solver の実装が 1 ファイル・1 関数 (solve_lp_dual_on) に手続き全体を書き下しているのに対し, HiGHS は「行の選択」「列の選択と比検定」「基底とその更新」を別クラスへ明確に分離し, さらに単体法・並列版 (SIP/PAMI)・MIP のノード再最適化など多くの呼び出し文脈を 1 つの HEkk 上で共有できるように設計されている。本稿はシリアル版 (kSimplexStrategyDualPlain, iterate()) の経路に絞って追う。

2 標準形とデータ構造

2.1 LP の内部表現: 範囲制約行と論理変数

HiGHS の LP は

$$\min_x c^\top x \quad \text{s.t.} \quad rl \leq Ax \leq ru, \quad cl \leq x \leq cu$$

という範囲制約 (ranged row) を直接扱える形で保持される (rl, ru, cl, cu はそれぞれ無限大を許す)。単体法内部では各行 i に対して論理変数 (logical variable) x_{n+i} を追加し,

$$A_{\text{ext}} = [A \mid I], \quad A_{\text{ext}} z = 0, \quad z = (x, s)$$

という斉次な等式標準形にする。行の可動域は論理変数側の上下限に符号反転して押し込まれる (HEkk::initialiseLpRowBound):

$$lb_{n+i} = -ru_i, \quad ub_{n+i} = -rl_i.$$

これにより, 等式行 ($rl_i = ru_i$), $\leq$ 行 ($rl_i = -\infty$), $\geq$ 行 ($ru_i = \infty$), 範囲行, 自由行 (両方無限大) のすべてが場合分けなしに同じデータ構造で表現される。ENOMOTO-Solver がモデル側で行

の種別 (`RowSense::Eq/Le/Ge`) ごとにスラック係数・上下限を作り分け、かつ範囲制約を持たないのと対照的である。

さらに構造変数 x_j 自身も両側無限大 (自由変数) を許す。これにより非基底変数の状態は「下限に張り付き」「上限に張り付き」に加えて「自由 (非基底のまま値 0)」の 3 通りになり、双対実行可能性の定義や比検定に特別な扱い (後述の `Freelist`) が必要になる。ENOMOTO-Solver は変数を必ず両側有限に制限することでこの場合分けを最初から消している。

2.2 基底：3 本の配列 `SimplexBasis`

```
struct SimplexBasis {
    std::vector<HighsInt> basicIndex_; // 行 i の基底変数番号
    std::vector<int8_t> nonbasicFlag_; // 0: 基底, 1: 非基底
    std::vector<int8_t> nonbasicMove_; // 非基底変数の「動く向き」
};
```

`basicIndex_` は ENOMOTO-Solver の `Tableau::basis` に対応する (`basicIndex_[i] = B` の i 番目の変数)。`nonbasicFlag_` は `basis_pos` の代わりに「基底か否か」だけを $O(1)$ で持つブール配列であり、行番号自体は持たない (非基底変数がどの行にいるかという情報自体が不要なため)。`nonbasicMove_` は ENOMOTO-Solver の `NbStatus::{Lower,Upper}` に対応するが、値は

$$\text{nonbasicMove}_-[j] \in \underbrace{\{-1\}}_{\text{上限}}, \underbrace{0}_{\text{固定/自由}}, \underbrace{+1}_{\text{下限}}$$

の 3 値であり、固定変数 ($lb_j = ub_j$) と自由変数がどちらも 0 に落ちる点、「非基底なら値がどちらの限界か」を符号だけで表す点が異なる。

2.3 作業配列 `HighsSimplexInfo` (`SimplexStruct.h`)

```
struct HighsSimplexInfo {
    std::vector<double> workCost_, workDual_, workShift_;
    std::vector<double> workLower_, workUpper_, workRange_, workValue_;
    std::vector<double> baseLower_, baseUpper_, baseValue_;
    ...
};
```

数式との対応：

`workCost_` $\leftrightarrow c$, `workDual_` $\leftrightarrow d$ (被約費用), `workLower_/workUpper_` $\leftrightarrow lb, ub$ (全変数, 列インデックス)

`workShift_` は退化対策としてコストへ加えられた一時的なシフト量 (後述の `shiftCost/shiftBack`) で、`workDual_` 自体には常にシフト後の値が入っている。`workValue_` は非基底変数の現在値 x_j ($j \in \mathcal{N}$) で、ENOMOTO-Solver の `Tableau::x` (非基底成分) に対応する。

特徴的なのは `baseLower_/baseUpper_/baseValue_` の存在である。これは行インデックスで見えた基底変数の上下限・値

$$\text{baseLower}_-[i] = lb_{\text{basicIndex}_-[i]}, \quad \text{baseUpper}_-[i] = ub_{\text{basicIndex}_-[i]}, \quad \text{baseValue}_-[i] = x_{\text{basicIndex}_-[i]}$$

を `basicIndex_` 経由の間引きなしにキャッシュしたもので、CHUZR が反復のたびに m 回行うであろう間接参照を消している。ENOMOTO-Solver の `Tableau::x` は列インデックス (全変数共通) 1 本で持つのに対し、HiGHS は「列インデックスで見た配列」と「行インデックスで見た基底専用キャッシュ」を意図的に二重化している。

2.4 DSE 重みと HEkkDualRow/HEkkDualRHS の作業領域

- `HEkk::dual_edge_weight_`: 行インデックスの双対 steepest edge 重み $w_i \approx \|B^{-\top} e_i\|^2$ (ENOMOTO-Solver `DseState::w` と同じ量)。Devex 使用時は `devex_index_` に切り替わる。
- `HEkkDualRow::packIndex/packValue`: ピボット行を「構造変数側 `row_ap`」と「論理変数側 `row_ep`」の両方をオフセットで連結して 1 本にまとめた疎表現 (`chooseMakepack`)。ENOMOTO-Solver の `a_p/touched_cols` に相当するが、論理変数の列を「 ρ_p そのもの」として物理的に連結する点異なる (HiGHS の論理変数の列は e_i なので $\rho_p^\top e_i = \rho_{p,i}$, つまり `row_ep` の値そのものになる)。
- `HEkkDualRow::workData/workGroup`: BFRT 候補 ($j, |\alpha_j|$) の配列と、退化候補をまとめる「グループ」境界インデックス (後述)。
- `HEkkDualRow::freeList`: 非基底の自由変数の集合 (`std::set`)。CHUZC の間だけ `nonbasicMove_` を一時的に符号確定させ (`createFreemove`)、計算が終われば 0 に戻す (`deleteFreemove`)。
- `HEkkDualRHS::work_infeasibility/workIndex/workMark/workCutoff`: CHUZR 用のハイパースパース候補リストとその足切り値 (後述)。

3 フェーズ 1 / フェーズ 2

3.1 開始前の判定: `HEkkDual::solve`

`solve` はまず摂動なしの費用で `computeDual` を呼び、双対実行可能かどうかを見る。実行可能または「わずかな違反のみ」ならフェーズ 2 を強制する (`force_phase2`)。これは ENOMOTO-Solver の `crash_dual_feasible` が常に無条件で双対実行可能な初期基底を作れるのと対照的で、HiGHS は自由変数・すでに非基底な変数など任意の初期基底から出発しうするため、双対実行可能性が保証されない一般の場合に備えてフェーズ 1 が要る。

3.2 フェーズ 1: 人工上下限による「双対不能性の一次不能性への変換」

双対実行可能でない場合、`HEkk::initialiseBound(kDual, kSolvePhase1)` が非基底変数ごとに一時的な人工上下限を設定する (実装は `HEkk.cpp`):

$$(lb_j, ub_j) \leftarrow \begin{cases} (-1000, 1000) & lb_j = -\infty \text{ かつ } ub_j = \infty \text{ (自由)} \\ (-1, 0) & lb_j = -\infty \text{ (上限のみ)} \\ (0, 1) & ub_j = \infty \text{ (下限のみ)} \\ (0, 0) & \text{両側有限 (箱型/固定)} \end{cases}$$

このもとで通常のフェーズ 2 と同じ反復 (`iterate()`) を回す。人工上下限は「その変数の現在の被約費用が双対実行可能な符号ならその境界での値が 0, そうでなければ実行不可能な向きへ ± 1 (自由変数は ± 1000)」となるように選ばれているため、このフェーズ 1 の (一次) 目的関数値

$$z_1 = \sum_{j \in \mathcal{N}} d_j x_j$$

は「双対実行不能量の総和の符号反転」に一致する (`solvePhase1` 冒頭のコメント)。したがって $z_1 = 0$ に達すればフェーズ 2 へ、非零のまま最適化が止まれば真の双対不能性 (`assessPhase1Optimality`) と判定する。

ENOMOTO-Solver はこの種のフェーズ 1 を持たない：全変数が両側有限という前提のもとで `crash_dual_feasible` が常に無条件成功するため、最初から「本物の」上下限のまま双対実行可能な基底を構成でき、あとはコスト摂動だけで退化に備えれば十分だからである。HiGHS のフェーズ 1 は、むしろ単体法のフェーズ 1 と双対な発想 (実行不能性を人工的な目的関数の最小化に変換する) を双対法側に適用したものと言える。

3.3 コスト摂動：HEkk::initialiseCost

退化対策として、フェーズ 2 (および必要ならフェーズ 1) に入る前に費用ベクトルを摂動する。

$$\bar{c} = \begin{cases} \max_j |c_j|^{1/4} & \max_j |c_j| > 100 \\ \max_j |c_j| & \text{それ以外} \end{cases}, \quad \text{base} = \mu \cdot 5 \times 10^{-7} \cdot \bar{c}$$

($\mu = \text{dual_simplex_cost_perturbation_multiplier}$)。「箱型」列の割合が 1% 未満なら $\bar{c} \leftarrow \min(\bar{c}, 1)$ とする点も含め、ENOMOTO-Solver の `perturb_costs` とほぼ同じ設計 (実装コメントにも HiGHS を参考にした旨が明記されている)。摂動量は列ごとに

$$\text{xpert}_j = (1 + r_j)(|c_j| + 1) \cdot \text{base}, \quad r_j \in [0, 1) \text{ (擬似乱数)}$$

で、自由列・固定列は摂動なし、片側有限列はその有限でない側へ、両側有限列は自分の費用の符号方向へ加える。加えて論理変数 (行) 側にも極小な摂動 $\mu \cdot 10^{-12}$ を独立に与える点は ENOMOTO-Solver にはない追加要素である。

4 メインループ：HEkkDual::iterate

1 反復の呼び出し順序は次の通り (各メソッドは冒頭で `if (rebuild_reason) return;` を持ち、途中で異常を検知すると即座に抜けて `rebuild()` に戻る設計)。

`chooseRow` $\rightarrow$ `chooseColumn` $\rightarrow$ `updateFtranBFRT` $\rightarrow$ `updateFtran` $\rightarrow$ `updateFtranDSE`
 $\rightarrow$ `updateVerify` $\rightarrow$ `updateDual` $\rightarrow$ `updatePrimal` $\rightarrow$ `updatePivots`

4.1 CHUZR：chooseRow と HEkkDualRHS::chooseNormal

一次実行不能な基底行の中から、DSE 重みで正規化した「見込み値 (merit)」

$$\text{merit}_i = \frac{\text{infeas}_i}{w_i}, \quad \text{infeas}_i = \max(lb_{\text{basicIndex}_-[i]} - x_i, x_i - ub_{\text{basicIndex}_-[i]}, 0)$$

を最大化する行を選ぶ点は ENOMOTO-Solver の `chuzr` と同じ発想である。ただし実装には次の違いがある。

- ハイパースパース候補リストに足切り値を導入：`createInfeasList` は、実行不能行数が多い場合、`nth_element` で「上位 k 件相当」の `merit` のしきい値 `workCutoff` を近似的に求め、それ未満の行を候補リストから除外する（完全なソートを避けた近似的な絞り込み）。以後 `updateInfeasList` は $\text{work_infeasibility}_i > \text{workCutoff} \cdot w_i$ を満たす行だけをリストに追加する。
- 走査開始点をランダム化：`chooseNormal` は候補リストの中央から始めて 2 区間に分けて走査する（`randomStart` 起点, 2 セクション）。これは「同点のとき常に先頭が勝つ」ことで生じる反復パターンの偏り（ENOMOTO-Solver では行番号を第二キーにした決定的なタイプレックで対処）を、決定論を捨てて乱数で崩す設計である（ただし乱数列自体は再現可能なシード付き）。
- **DSE** 重みの事後検証と再選択：選んだ行についていったん BTRAN で $\rho_p = B^{-\top} e_p$ を計算し、そこから「真の」重み $\|\rho_p\|^2$ を計算し直して、増分更新してきた重みと比較する（`acceptDualSteepestEdgeWeight`）。更新値が真の値の `kAcceptDseWeightThreshold` 倍未満ならその行を棄却して再度 *CHUZR* からやり直す。ENOMOTO-Solver は増分更新した重みを無条件に信頼する。

行 p が確定すると、退出変数 `variable_out = basicIndex_[p]`, 退出方向 `move_out` $\in \{-1, +1\}$ （下限割れなら -1 , 上限超過なら $+1$ ）、及び `delta_primal = xvariable_out - {lb or ub}` が記録される。

4.2 PRICE と CHUZO: `chooseColumn` と `HEkkDualRow`

■PRICE. `ekk_instance_.tableauRowPrice` が行毎に格納された非基底の行列（`ar_matrix_`, 構造変数の行持ち表現）を使って

$$a_p^\top = \rho_p^\top A$$

を計算する（`row_ap`）。ENOMOTO-Solver の PRICE（ ρ_p の非零行だけ走査）と同じ疎行列積だが、HiGHS は列持ち `cols` ではなく専用の行持ち疎行列 `ar_matrix_` を保守している。

■パッキングと自由変数の一時固定. `dualRow.chooseMakepack` が `row_ap`（構造変数）と `row_ep`（論理変数, オフセット `num_col_`）を 1 本の疎配列（`packIndex`, `packValue`）にまとめる。`createFreemove` は非基底の自由変数 $j \in \text{freeList}$ について、このピボット行に対する成分 $\alpha_j = a_p^\top A_j$ の符号から `nonbasicMove_[j]` を一時的に確定させ、自由変数の被約費用がこの反復で「実行不能」と誤判定されるのを防ぐ。ENOMOTO-Solver には自由変数が存在しないためこの仕組み自体が不要である。

■`choosePossible`（一次フィルタ）. 退出方向 `move_out` と各候補の `nonbasicMove_` から適格性を判定し（ENOMOTO-Solver の `chuzc1` の符号条件と同じ考え方）、粗い上界

$$\text{workTheta} = \min_j \frac{d_j \cdot \text{nonbasicMove}_[j] + T_d}{|\alpha_j|}$$

を一次候補全体から見積もる。

■**chooseFinal** (BFRT 本体) . ENOMOTO-Solver が候補全体を 比の昇順で完全にソート／ヒープ化するのに対し, HiGHS は次の 3 段階で「全件ソートを避けた」比検定を行う。

1. 大まかな足切り : **selectTheta** を **workTheta** の 10 倍から始め, 10 倍ずつ拡大しながら, $\text{tight}_j := \text{nonbasicMove}_j \cdot d_j \leq \alpha_j \cdot \text{selectTheta}$ を満たす候補だけを残す。累積

$$\text{totalChange} = \sum_{j \text{ 残存}} \text{workRange}_j \cdot \alpha_j$$

が退出変数の必要移動量 $\text{totalDelta} = |\text{delta_primal}|$ 以上になるか, 候補を使い切ったら打ち切る。

2. グループ化 (**chooseFinalWorkGroupQuad**) : 残った候補を, **selectTheta** を「まだ厳しくない候補の中で最も近い比」へ逐次更新しながら「同じ **selectTheta** 帯に入る」候補ごとにグループへ束ねる (**workGroup** が各グループの境界)。ENOMOTO-Solver の「昇順に 1 件ずつ確定しながら退出変数の目標値に達するまで歩く」処理と目的は同じだが, 同点 (退化) 候補を 1 グループにまとめて扱う点異なる。**totalChange** が **totalDelta** に達したグループが, 退出変数をちょうど目標値まで運ぶのに必要な最後のグループ (**breakGroup**) である。
3. 大ピボット選択 (**chooseFinalLargeAlpha, long-step** 比検定) : **breakGroup** から番号の大きい方 (比の大きい方) へ逆向きにグループを走査し, 各グループ内の最大ピボット $|\alpha|$ が

$$|\alpha| > \text{finalCompare} := \min(0.1 \cdot \max_j |\alpha_j|, 1)$$

を満たす最初のグループを見つけたら, そのグループ内の最大ピボットの候補を真の進行列 q として確定する。見つからなければ 1 つ手前のグループへ進む。これは ENOMOTO-Solver の「**stop_idx** から後方へ **HARRIS_RATIO_TOL** 幅の窓を探して最大ピボットを選ぶ」処理と数値的安定性の狙いは同じだが, 固定の許容誤差幅ではなく「グループ単位・相対 10%」という基準を使う点異なる。

確定した $q = \text{workPivot}$, $\alpha_{\text{row}} = \text{workAlpha}$, 双対ステップ $\text{theta_dual} = \text{workTheta} = d_q / \alpha_{\text{row}}$ が **HEkkDual** 側に返される。**breakGroup** より手前の (= より小さい比の) 候補群は全件フリップ対象として **workData** に積み直され (ステップ 4, $\text{workData}_i = (\text{col}, \text{nonbasicMove}_i \cdot \text{workRange}_i)$), これが後続の **updateFlip** で実際に反転される。

4.3 FTRAN 群 : **updateFtranBFRT** / **updateFtran** / **updateFtranDSE**

$$\begin{aligned} \text{BFRT 合成列 : } B \text{ col_BFRT} &= \sum_{j:\text{flip}} A_j \cdot (\text{nonbasicMove}_j \cdot \text{workRange}_j) \\ &\quad (\text{dualRow.updateFlip} + \text{ftran}) \\ \text{進行列 : } B \text{ col_aq} &= A_q \quad (\alpha_{\text{col}} = \text{col_aq}[p]) \\ \text{DSE 交差項 : } B \tau &= \rho_p \quad (\text{updateFtranDSE, DSE 使用時のみ}) \end{aligned}$$

ENOMOTO-Solver の `combined_alpha_buf` / `alpha_buf` / `tau_buf` にそれぞれ対応する。HiGHS は `updateFtranBFRT` を「`dualRow.workCount==0` なら FTRAN 自体をスキップする」ようガードしており、フリップが 1 件も無い（非退化な）反復ではこのコストがゼロになる。

4.4 数値検証：updateVerify

進入列を列方向 FTRAN で求めたピボット `alpha_col` と、PRICE（行方向）で求めたピボット `alpha_row` を比較し、相対誤差が大きければ `rebuild_reason = kRebuildReasonPossiblySingularBasis` を立てて即座に再分解へ回す。ENOMOTO-Solver には行・列 2 通りで同じピボットを求め相互検証するステップはなく（FTRAN 一度きり）、代わりに一定間隔ごとの残差ノルムチェック（トリガー 1）で基底のドリフトを検知する。

4.5 双対値の更新：updateDual

`dualRow.updateDual(theta_dual)` が、パックされたピボット行の全成分に対して

$$d_j \leftarrow d_j - \text{theta_dual} \cdot a_{p,j} \quad (j \in \text{パック済み列})$$

を適用する。これは ENOMOTO-Solver の `update-dual` と同一の閉形式である。`theta_dual=0`（退出変数がすでに退化的にちょうど境界にある場合）のときは全列更新をスキップし、進入変数のみ `shiftCost` で一時的なコストシフトを与えて `workDual[q] = 0` を保つ（本質的に d_q をシフトでゼロに固定する簡便策）。基底を出る変数については更新後に `shiftBack` で以前のシフトを取り除く。

4.6 主変数と DSE/Devex 重みの更新：updatePrimal

BFRT フリップ分（`col_BFRT`）をまず `dualRHS.updatePrimal` で反映し、続いて主ステップ幅

$$\text{theta_primal} = \frac{x_{\text{variable_out}} - \{lb \text{ or } ub\}}{\text{alpha_col}}$$

で `col_aq` 分を反映する（ENOMOTO-Solver の θ_q と同一の式）。DSE 使用時は

$$w_i \leftarrow w_i + \underbrace{\alpha_i \left(\frac{w_p}{\alpha_p^2} \right)}_{\text{新しい } w_q} \cdot \alpha_i + \underbrace{\left(-\frac{2}{\alpha_p} \right)}_{\text{Kai}} \cdot \tau_i = w_i - 2 \left(\frac{\alpha_i}{\alpha_p} \right) \tau_i + \left(\frac{\alpha_i}{\alpha_p} \right)^2 w_p$$

という更新（`HEkk::updateDualSteepestEdgeWeights`）を行う——最右辺の形は ENOMOTO-Solver の `DseState::update_after_pivot` の式と厳密に一致しており、両者が同じ Forrest – Goldfarb 型階数 1 更新公式に依拠していることを裏付ける。Devex 使用時は `updateDualDevexWeights` が同様の役割を近似的な重み（厳密な steepest edge ではなく参照集合ベースの近似）で行う。

4.7 基底交換：updatePivots

`ekk_instance_.updatePivots` が `nonbasicFlag_/basicIndex_/nonbasicMove_` を書き換え、`updateFactor` が invertible 表現（HiGHS 自身の LU + eta 更新エンジン `HFactor`

／HSimplexNla)を更新し、updateMatrix が次回 PRICE 用の行持ち非基底行列を更新する。dualRow.deleteFreelist(variable_in) が進入変数を自由変数リストから除く。最後に dualRHS.updatePivots が退出行の新しい基底値とその一次実行不能性を書き込む。

5 Devex と DSE の切り替え, および安定化機構のまとめ

HiGHS は既定で DSE を使うが, 反復が進んで重み誤差が蓄積した場合 (newDevexFramework) や, コスト構造上 DSE が割に合わない判断された場合には Devex 法へ切り替える機構 (interpretDualEdgeWeightStrategy, possiblyUseLiDualSteepestEdge) を持つ。ENOMOTO-Solver は DSE 固定である。

退化・数値安定化の課題	HiGHS (HEkkDual)	ENOMOTO-Solver (simplex.rs)
初期基底が双対不能なとき	人工上下限によるフェーズ 1 (本物の一次単体法反復)	発生しない (両側有限の前提で crash が常に成功)
被約費用の退化 (ties)	コスト摂動 (列ごとの向き付き摂動)	同種のコスト摂動 (HiGHS を参考に実装)
BFRT の数値安定性	グループ化 + long-step large-alpha 選択	昇順ヒープ + 後方 Harris 窓探索
CHUZR の退化	走査開始点のランダム化	行番号によるタイブレーク (決定的)
DSE 重みの誤差蓄積	事後検証つき再選択 + Devex へのフォールバック	検証なし (単純増分更新のみ)
ピボットの数値検証	行/列 2 通りの計算値を比較 (updateVerify)	残差ノルム/最小ピボット/eta 蓄積の 3 トリガー
基底特異への対処	直近の非特異基底へバックトラック	主単体法へフォールバック
最後の手段の反サイクリング	(フェーズ 1 の反復上限 + 摂動)	Bland の最小添字規則

6 データ構造のまとめ

クラス／フィールド	数式上の対象	備考
SimplexBasis{basicIndex_,nonbasicFlag_,nonbasicMove_}	$\mathcal{B}$, 所属フラグ, 非基底の張り付き方向 (自由/固定含む 3 値)	3 配列に分割
HighsSimplexInfo::{workCost_,workDual_,workShift_}	c, d , 一時コストシフト	workDual_ は常にシフト後
HighsSimplexInfo::{workLower_,workUpper_,workValue_}	lb, ub (全変数), 非基底値 x_j	列インデックス

クラス／フィールド	数式上の対象	備考
HighsSimplexInfo::{baseLower_,baseUpper_,baseValue_}	基底変数の lb, ub, x	行インデックスへの再キャッシュ
HEkk::dual_edge_weight_	$w_i \approx \ B^{-\top} e_i\ ^2$	Devex 時は devex_index_
HEkkDualRow::{packIndex,packValue}	ピボット行 a_p (構造 + 論理変数を連結)	PRICE の出力
HEkkDualRow::{workData,workGroup}	BFRT 候補と退化グループ境界	グループ化 BFRT
HEkkDualRow::freeList	非基底の自由変数集合	CHUZC 中のみ move 確定
HEkkDualRHS::{work_infeasibility,workIndex,workCutoff}	一次実行不能行の候補集合と足切り値	ハイパースペース CHUZR

7 まとめ

HiGHS の HEkkDual は, ENOMOTO-Solver と同じ「BTRAN → PRICE → CHUZR/CHUZC → FTRAN → 基底交換」という双対単体法の骨格を共有しつつ, (1) 論理変数を用いた斉次標準形により範囲制約・自由変数を場合分けなしに表現し, (2) 双対不能な初期基底からでも人工上下限によるフェーズ 1 で出発できるようにし, (3) BFRT をソートではなくグループ化と long-step large-alpha 選択で行うことで大規模問題での比検定コストを抑え, (4) DSE 重みの事後検証と Devex へのフォールバックで重み誤差の蓄積に備え, (5) 行・列 2 通りのピボット値比較と特異基底からのバックトラックで数値的健全性を担保する, という, より汎用かつ多層的な安定化を備えた実装になっている。ENOMOTO-Solver の実装は「全変数が両側有限」という強い前提を活かしてこれらの多くを単純化・省略した設計であり, 両者を比較することで, どの安定化機構が「有界変数のみ」という前提のもとでは不要になるかが明確になる。